

Capacité des modèles de langage à générer des treillis de concepts et des règles d'implication



Auteurs :

Anthony Hommais
Corentin Fay

Encadrants :

Marianne Huchard
Yulin Zhang

Lien Github :

https://github.com/CorentinFW/LLM_Et_Treillis_2026

Université de Montpellier, faculté des Sciences :

Master 1 Génie Logiciel

mai 2026

Table des matières

1	Introduction	5
2	Fondements théoriques de l'analyse formelle de concepts	6
2.1	Contexte formel	7
2.2	Opérateurs de dérivation	7
2.3	Concepts formels	7
2.4	Treillis de concepts	8
2.5	Calcul des fermetures, génération des concepts et du treillis	8
2.6	Complexité et passage à l'échelle	9
3	État de l'art sur la génération de code par LLM	10
3.1	Génération de code par modèles de langage	10
3.2	Validation et comparaison d'algorithmes générés	10
3.3	Limites des benchmarks existants	11
4	Spécifications et définition de la correction	11
4.1	Format d'entrée des contextes formels	12
4.2	Format de sortie des treillis	12
4.3	Variante de représentation	13
4.4	Définition opérationnelle de la correction	14
4.5	Cas non comparables et statuts d'exécution	14
5	Méthodologie expérimentale	15
5.1	Implémentation de référence	15
5.2	Implémentations générées par les LLM	15
5.3	Choix des modèles de langage	16
5.4	Jeux de données	17
5.4.1	Jeux synthétiques	17
5.4.2	Jeux réels binarisés	17
5.5	Pipeline expérimental	17
5.5.1	Exécution	18
5.5.2	Normalisation	18
5.5.3	Comparaison structurelle	18
5.5.4	Génération des rapports	18
5.6	Mesures de performance et de ressources	18
5.6.1	Temps d'exécution	18
5.6.2	Wrappers de mesure des ressources	18
5.6.3	Consommation mémoire	19
5.6.4	Utilisation disque et I/O	19
5.7	Reproductibilité	19
5.7.1	Modèles de langage utilisés	19
5.7.2	Analyse du prompt utilisé dans les implémentations sans contrainte mémoire	20
5.7.3	Analyse du prompt utilisé dans les implémentations avec contrainte mémoire	21
5.8	Cartographie et organisation du dépôt	23
5.9	Environnement matériel et logiciel	23
5.9.1	Système d'exploitation	23
5.9.2	Configuration matérielle	24

6	Implémentations générées	24
6.1	Implémentation classique sans contrainte mémoire	24
6.1.1	Structure générale	24
6.1.2	Structures de données	25
6.1.3	Implémentation de Next Closure	25
6.1.4	Calcul du diagramme de Hasse	25
6.1.5	Export Graphviz	25
6.1.6	Influence du prompt sur l'implémentation	26
6.1.7	Analyse de l'approche	26
6.2	Implémentation orientée efficacité mémoire	26
6.2.1	Principes généraux	26
6.2.2	Représentation par bitmasks	27
6.2.3	Partitionnement des concepts	27
6.2.4	Calcul adaptatif de la taille des lots	27
6.2.5	Calcul de la relation de couverture	27
6.2.6	Génération du fichier DOT	28
6.2.7	Influence du prompt sur l'implémentations	28
6.2.8	Analyse de l'approche	28
6.3	Comparaison des deux approches	28
7	Résultats expérimentaux	29
7.1	Résultats de correction structurelle	29
7.1.1	Implémentations sans contrainte mémoire	29
7.1.2	Implémentations orientées économie mémoire	30
7.2	Résultats de performance	30
7.2.1	Temps d'exécution	30
7.2.2	Analyse par famille de datasets	30
7.3	Scalabilité	31
7.4	Consommation des ressources	31
7.4.1	Consommation mémoire	31
7.4.2	Écritures disque	31
7.5	Robustesse	31
8	Analyse des erreurs et discussion	32
8.1	Interprétation générale des résultats	32
8.2	Causes principales d'échec	32
8.3	Comportements émergents des implémentations	33
8.4	Impact indirect des contraintes de prompt	33
8.5	Compromis algorithmiques	33
8.6	Synthèse	33
9	Menaces à la validité (Threats to validity)	34
9.1	Validité interne	34
9.2	Validité externe	34
9.3	Biais méthodologiques	35
9.4	Synthèse	35
10	Conclusion et perspectives	35

A	Annexe	37
A.1	Prompt	37
A.1.1	Prompt sans contrainte	37
A.1.2	Prompt avec contrainte mémoire	39

1 Introduction

Les grands modèles de langage (*Large Language Models*, LLM) occupent aujourd’hui une place importante dans les outils d’assistance à la programmation. Leur capacité à générer du code source à partir d’instructions en langage naturel a conduit à une adoption croissante dans des contextes variés, allant de l’automatisation de tâches simples à l’assistance au développement logiciel complexe. Cependant, si de nombreux travaux évaluent les LLM sur des tâches de génération de code classiques, peu d’études analysent leur capacité à produire des implémentations correctes d’algorithmes possédant des propriétés mathématiques et structurelles fortes et construisant des objets avec une combinatoire élevée.

Un cadre intéressant pour mener de telles analyses est celui de l’analyse formelle de concepts (AFC ou FCA, [7]). La FCA est un domaine issu de l’analyse de données et des mathématiques discrètes permettant de représenter des relations entre objets et attributs sous forme de structures ordonnées appelées *treillis de concepts* afin d’en extraire de la connaissance. La construction de tels treillis repose sur des propriétés algorithmiques non triviales : calculs de fermetures, énumération de concepts, gestion des relations de couverture et maintien d’invariants structurels. Ces propriétés rendent la génération correcte d’un algorithme construisant un treillis de concepts particulièrement exigeante pour un modèle de langage.

Ce travail a été réalisé dans le cadre d’un *Travail d’Étude et de Recherche* (TER) proposé par l’équipe de recherche SLICE du LIRMM. Le projet a été mené par deux étudiants autour d’un objectif commun : étudier dans quelle mesure des modèles de langage sont capables de générer automatiquement des implémentations d’algorithmes pour le calcul de structures dans le domaine de l’analyse formelle de concepts (*Formal Concept Analysis*, ou FCA). Il prend la suite d’un travail de TER de l’année dernière dans lequel des limites avaient été rencontrées dans la génération [5].

Deux approches sont explorées, qui diffèrent par leurs contraintes et leurs objectifs expérimentaux. La première approche vise principalement à obtenir des implémentations fonctionnellement correctes capables de produire un treillis valide à partir d’un contexte formel. La seconde approche, qui constitue une part importante de ce travail, s’intéresse plus spécifiquement à la génération d’algorithmes dits *memory-aware*, c’est-à-dire conçus pour limiter la consommation mémoire lors du calcul des treillis. Cette problématique de consommation mémoire est particulièrement pertinente dans le contexte de la FCA. En effet, le calcul de treillis de concepts peut conduire à une explosion combinatoire importante, tant sur le nombre de concepts générés que sur les structures intermédiaires manipulées. Certaines implémentations de référence, comme FCA4J¹ [8], peuvent ainsi rencontrer des limitations liées à l’utilisation de la mémoire vive lors du traitement de grands contextes formels. L’étude de stratégies visant à réduire cette consommation mémoire constitue donc un enjeu algorithmique important.

L’objectif de cette étude n’est donc pas uniquement d’évaluer la capacité des LLM à produire du code exécutable, mais également leur capacité à générer des implémentations structurellement correctes, robustes et capables de passer à l’échelle. Dans ce contexte, un programme produisant une sortie syntaxiquement valide mais un treillis incorrect ne peut pas être considéré comme satisfaisant. De même, une implémentation correcte mais incapable de gérer efficacement les ressources mémoire sur des jeux de données plus importants présente des limites pratiques importantes.

Afin d’évaluer rigoureusement les implémentations générées, nous avons conçu un protocole expérimental reproductible reposant sur :

- une spécification stricte des formats d’entrée et de sortie,
- une implémentation de référence basée sur FCA4J,
- une procédure de normalisation des treillis générés,
- un système de comparaison entre les sorties des programmes générés et l’implémentation de

1. <https://www.lirmm.fr/fca4j/>

- référence par équivalence structurelle,
- un pipeline automatisé de mesure des performances et des ressources.

Les implémentations produites par plusieurs modèles de langage ont été évaluées sur différents jeux de données synthétiques et réels binarisés. Les expériences portent à la fois sur :

- la correction des treillis générés,
- les performances temporelles,
- l'utilisation de mémoire vive et disque,
- la robustesse face à des cas limites,
- la capacité de montée en charge.

Les contributions principales de ce travail sont les suivantes :

- la définition d'un protocole de validation structurelle pour des treillis de concepts générés automatiquement,
- la conception d'un pipeline expérimental modulaire et reproductible,
- l'analyse comparative d'implémentations générées par différents LLM,
- l'étude de stratégies algorithmiques visant à réduire la consommation mémoire lors du calcul de treillis de concepts,
- une analyse des limites actuelles des modèles de langage pour la génération d'algorithmes combinatoires formels.

La suite de ce rapport est organisée comme suit. La section 2 introduit les notions théoriques nécessaires relatives à FCA. La section 3 présente l'état de l'art la génération de code par LLM et les méthodes d'évaluation existantes. La section 4 décrit les spécifications des formats et les critères de validation retenus. La section 5 détaille la méthodologie expérimentale et le pipeline de comparaison. La section 6 présente les différentes implémentations étudiées, les prompts utilisés ainsi que les stratégies de prompting employées pour guider la génération des programmes. La section 7 expose les résultats expérimentaux obtenus, tant sur les aspects de correction structurelle que sur les performances temporelles, mémoire et disque. La section 8 propose une analyse des erreurs observées, des comportements émergents des modèles et des compromis identifiés entre correction, performances et contraintes mémoire. La section 9 discute des menaces à la validité de l'étude, notamment les limites méthodologiques, les biais expérimentaux et les difficultés de généralisation des résultats. Enfin, la section 10 conclut ce travail en synthétisant les principaux enseignements obtenus et en présentant plusieurs perspectives de recherche futures.

2 Fondements théoriques de l'analyse formelle de concepts

Cette section présente les notions théoriques nécessaires à la compréhension des algorithmes étudiés dans ce travail. Nous introduisons les définitions fondamentales de *l'analyse formelle de concepts* (appelée *FCA* dans ce rapport), les opérateurs de dérivation, les concepts formels ainsi que la structure de treillis associée. Ces éléments constituent la base mathématique des implémentations générées et des critères de validation utilisés dans notre protocole expérimental.

L'analyse formelle de concepts (FCA), introduite notamment par Ganter et Wille [7], est une méthode mathématique d'analyse de données fondée sur la théorie des treillis et plus particulièrement des treillis de Galois [3]. Elle permet de représenter les relations entre des objets et des attributs (contextes formels) sous la forme de concepts formels organisés dans un treillis ou d'autres structures d'ordres partielles, ou encore sous forme de règles d'associations ou d'implication. Nous n'étudierons que la construction des treillis de concepts dans ce projet.

2.1 Contexte formel

Un contexte formel est défini par un triplet :

$$K = (G, M, I)$$

où :

- G est un ensemble fini d'objets,
 - M est un ensemble fini d'attributs,
 - $I \subseteq G \times M$ est une relation binaire appelée relation d'incidence.
- Pour un objet $g \in G$ et un attribut $m \in M$, la relation :

$$(g, m) \in I$$

signifie que l'objet g possède l'attribut m .

Dans la pratique, un contexte formel peut être représenté sous forme d'une matrice binaire où :

- les lignes correspondent aux objets,
- les colonnes correspondent aux attributs,
- une valeur 1 indique la présence d'un attribut pour un objet donné.

Cette représentation est celle utilisée dans les jeux de données manipulés dans notre étude.

2.2 Opérateurs de dérivation

La FCA repose sur deux opérateurs de dérivation permettant d'associer :

- à un ensemble d'objets les attributs qu'ils partagent,
- à un ensemble d'attributs les objets qui les possèdent.

Pour un ensemble d'objets $A \subseteq G$, on définit :

$$A' = \{m \in M \mid \forall g \in A, (g, m) \in I\}$$

L'ensemble A' correspond donc aux attributs communs à tous les objets de A .

Symétriquement, pour un ensemble d'attributs $B \subseteq M$, on définit :

$$B' = \{g \in G \mid \forall m \in B, (g, m) \in I\}$$

L'ensemble B' représente les objets possédant tous les attributs de B .

L'application successive de ces opérateurs définit les opérateurs de fermetures (familièrement appelés “fermetures” :

$$A'' = (A')'$$

et (de manière équivalente)

$$B'' = (B')'$$

Ces opérateurs de fermeture jouent un rôle central dans les algorithmes de génération de concepts.

2.3 Concepts formels

Un concept formel est un couple :

$$(A, B)$$

tel que :

$$A \subseteq G, \quad B \subseteq M$$

et vérifiant :

$$A' = B$$

et

$$B' = A$$

Dans ce couple :

- A est appelé l'extension (*extent* en anglais) du concept,
- B est appelé l'intension² (*intent* en anglais) du concept.

Intuitivement :

- l'extension représente les objets appartenant au concept,
- l'intension représente les attributs caractérisant ce concept.

L'ensemble de tous les concepts formels d'un contexte peut être organisé selon une relation d'ordre.

2.4 Treillis de concepts

Les concepts formels sont ordonnés par inclusion des extensions, ou de manière équivalente par inclusion inverse des intensions.

Pour deux concepts :

$$(A_1, B_1) \quad \text{et} \quad (A_2, B_2)$$

on définit :

$$(A_1, B_1) \leq (A_2, B_2) \iff A_1 \subseteq A_2$$

ce qui est équivalent à :

$$B_2 \subseteq B_1$$

Le treillis de concepts est l'ordre partiel résultant de l'ensemble des concepts muni cette relation de spécialisation.

On représente en général le treillis de concepts par son diagramme de Hasse. Dans cette représentation, chaque sommet représente un concept formel et chaque arête correspond à une relation de couverture entre deux concepts. Dans notre étude, les treillis sont représentés textuellement sous forme de graphes orientés au format Graphviz DOT. Les relations de couverture sont orientées de manière cohérente avec la convention :

$$\text{rankdir=BT}$$

afin de représenter graphiquement la hiérarchie des concepts avec les concepts les plus spécialisés plus bas que leurs généralisations.

2.5 Calcul des fermetures, génération des concepts et du treillis

La génération d'un treillis nécessite l'énumération de l'ensemble des concepts formels d'un contexte. Pour cela, la plupart des algorithmes FCA reposent sur le calcul de fermetures :

$$B'', \text{ avec } B \subseteq M$$

afin de produire des intensions (on parle aussi d'ensembles fermés pour la composition '' qui est une opération de fermeture) sans duplication. Intuitivement, un ensemble est dit fermé lorsque la

2. Ce terme d'intension, utilisé en philosophie dans le cadre de la description de concepts se distingue du terme habituel d'intention

réapplication de l'opération de fermeture ne produit pas un nouvel ensemble, i.e. si $B1$ est fermé, $B1'' = B1$.

Le calcul du treillis consiste ensuite à déterminer les relations de couverture de l'ordre de spécialisation.

2.6 Complexité et passage à l'échelle

Plusieurs algorithmes ont été proposés dans la littérature pour effectuer l'énumération des concepts et le calcul de la relation de couverture [10]. Parmi les approches classiques figurent notamment :

- *NextClosure*,
- *CloseByOne*,
- les variantes de la famille *In-Close*.

Ces algorithmes reposent généralement sur des calculs de fermeture (application successive des opérateurs α et β) et des stratégies d'énumération garantissant l'exploration exhaustive des concepts sans duplication. Cependant, la génération de treillis présente des difficultés algorithmiques importantes. Le nombre de concepts formels peut croître exponentiellement avec la taille du contexte, conduisant à une explosion combinatoire aussi bien en temps qu'en mémoire. Plus précisément, dans le pire des cas, il peut atteindre $2^{\min(|G|, |M|)}$, dans le cas où il est isomorphe à l'ensemble des parties du plus petit des deux ensembles G et M . Cette propriété rend le calcul de treillis particulièrement sensible aux choix de représentation des données, aux structures mémoire utilisées ainsi qu'aux stratégies de génération des relations de couverture.

Elle a des conséquences importantes :

- augmentation rapide du temps d'exécution,
- forte consommation mémoire,
- croissance importante des structures intermédiaires,
- augmentation du coût du calcul des relations de couverture.

En pratique, les performances dépendent fortement :

- de la taille du contexte,
- de la densité de la matrice binaire
- de la distribution des relations dans le contexte,
- du nombre réel de concepts générés,
- des structures de données utilisées.

Dans les implémentations pratiques, plusieurs stratégies sont utilisées afin de limiter ces coûts :

- représentations compactes par bitsets,
- calcul paresseux (*lazy evaluation*),
- partitionnement des données,
- externalisation partielle sur disque,
- réduction des structures intermédiaires conservées en mémoire.

Ces problématiques sont particulièrement importantes dans le cadre de notre étude, puisque nous chercherons à générer des implémentations dédiée explicitement à la réduction de la consommation mémoire par rapport à des implémentations plus classiques.

Ces contraintes expliquent l'importance des stratégies d'optimisation étudiées dans ce travail, notamment les approches visant à limiter l'utilisation de la mémoire vive par externalisation partielle des données ou réduction des structures conservées en mémoire.

3 État de l’art sur la génération de code par LLM

Cette section présente les principaux travaux et notions nécessaires pour situer notre étude dans le contexte actuel de la génération de code par des LLMs. Nous présentons ensuite les travaux récents sur la génération de code par modèles de langage ainsi que les limites des méthodes actuelles d’évaluation des systèmes de génération automatique de programmes.

3.1 Génération de code par modèles de langage

Les modèles de langage de grande taille sont désormais largement utilisés pour la génération automatique de code source. Des systèmes tels que GPT, Claude, Gemini ou encore les modèles open-source récents ont démontré leur capacité à produire des fonctions, scripts ou applications complètes à partir d’instructions rédigées en langage naturel. L’évaluation de ces modèles repose généralement sur des benchmarks dédiés à la génération de code. Parmi les plus connus figurent notamment :

- *HumanEval* : <https://github.com/openai/human-eval>,
- *MBPP* (*Mostly Basic Programming Problems*) : <https://github.com/google-research/google-research/tree/master/mbpp>,
- *SWE-bench* : <https://github.com/SWE-bench/SWE-bench>.

Ces benchmarks évaluent principalement la capacité des modèles à générer du code satisfaisant des tests unitaires ou corrigeant des problèmes logiciels identifiés. Les métriques utilisées portent souvent sur le taux de réussite aux tests, la compilation du code ou encore la capacité à produire une sortie attendue sur des cas donnés. Cependant, ces approches présentent plusieurs limites dans le contexte d’algorithmes mathématiques complexes. Dans de nombreux cas, un programme peut réussir des tests simples tout en restant incorrect du point de vue algorithmique ou structurel. Des travaux récents ont notamment montré que plusieurs benchmarks classiques de génération de code possèdent une couverture de tests insuffisante, permettant à des implémentations incorrectes de réussir les évaluations automatiques [11]. Par ailleurs, des benchmarks comme *HumanEval* ou *MBPP* restent majoritairement composés de problèmes relativement simples et peu représentatifs de tâches algorithmiques complexes ou de scénarios logiciels réels [15]. Enfin, ces évaluations mesurent principalement la capacité des modèles à produire du code passant des tests unitaires, sans réellement évaluer leur compréhension sémantique profonde, leur raisonnement structurel ou la qualité globale des implémentations produites.

Ils pèchent ainsi sur :

- le respect d’invariants mathématiques globaux,
- la validité structurelle des objets produits,
- la robustesse sur des cas limites,
- les propriétés de passage à l’échelle,
- les coûts mémoire et disque associés à l’exécution.

Le calcul de treillis de concepts constitue ainsi un cas d’étude particulièrement intéressant pour l’évaluation des capacités réelles des LLM en génération algorithmique et de code. La production d’un treillis correct nécessite non seulement du code exécutable, mais également le respect de propriétés structurelles fortes qui demandent des tests unitaires sur des cas variés et relativement complexes, ainsi que des vérifications soigneuses du résultat.

3.2 Validation et comparaison d’algorithmes générés

L’un des principaux défis liés à l’évaluation de code généré automatiquement réside dans la définition de critères de correction fiables. Dans le cas d’algorithmes produisant des structures complexes, une comparaison textuelle des sorties est généralement insuffisante. Dans le domaine

de la FCA, plusieurs représentations différentes peuvent en effet décrire un même treillis lorsqu'on se trouve en présence :

- d'un ordre différent des nœuds,
- d'identifiants locaux différents,
- de labels réduits ou complets,
- de variations syntaxiques dans la représentation textuelle (par exemple en DOT, format textuel largement utilisé pour représenter des graphes et manipulé par le logiciel Graphviz : <https://graphviz.org/>).

Une comparaison naïve des fichiers générés conduirait donc à de nombreux faux négatifs. Il devient nécessaire de comparer les structures produites à un niveau plus abstrait. Dans le cadre de cette étude, nous adoptons une approche fondée sur l'équivalence structurelle des treillis. Les sorties générées sont normalisées puis transformées en représentations canoniques permettant de comparer :

- les ensembles de concepts,
- les intensions et les extensions des concepts,
- les relations de couverture.

Cette approche permet d'évaluer la correction des implémentations indépendamment des variations purement syntaxiques des fichiers DOT générés.

3.3 Limites des benchmarks existants

Les benchmarks actuels de génération de code se concentrent majoritairement sur des tâches locales ou fonctionnelles [4, 2, 9] :

- génération de fonctions isolées,
- résolution de petits problèmes algorithmiques,
- correction de bugs,
- passage de tests unitaires.

À l'inverse, notre étude s'intéresse à un problème combinatoire nécessitant :

- la génération d'un algorithme complet,
- le maintien d'invariants théoriques globaux,
- la gestion de ressources importantes,
- la production d'une structure mathématique valide,
- une comparaison structurelle robuste des résultats avec une implémentation de référence.

Le protocole proposé cherche ainsi à dépasser une simple évaluation de "code fonctionnel" pour étudier la capacité des modèles de langage à produire des implémentations réellement correctes et exploitables dans un contexte algorithmique complexe.

4 Spécifications et définition de la correction

Afin d'évaluer rigoureusement les implémentations générées par les modèles de langage, il est nécessaire de définir précisément :

- les formats d'entrée et de sortie attendus,
- les contraintes de validité associées,
- les critères permettant de déterminer si une implémentation est correcte.

Dans le cadre de cette étude, la validation ne repose pas sur une simple comparaison textuelle

des fichiers produits, mais sur une comparaison structurelle des treillis générés.
 Cette section formalise les conventions utilisées tout au long du protocole expérimental.

4.1 Format d'entrée des contextes formels

Les contextes formels utilisés dans les expériences sont représentés sous forme de fichiers CSV binaires.

Chaque fichier décrit :

- un ensemble d'objets représentés par les lignes,
- un ensemble d'attributs représentés par les colonnes,
- une relation d'incidence binaire codée par des valeurs 0 et 1.

Le format attendu respecte les contraintes suivantes :

- la première case reste vide,
- la première ligne contient les noms des attributs,
- la première colonne contient les identifiants des objets,
- toutes les lignes doivent avoir la même largeur,
- les valeurs d'incidence doivent être strictement binaires (0 ou 1),
- les séparateurs , et ; sont autorisés,
- certains jeux de données peuvent contenir des champs entre guillemets.

Le tableau 1 illustre un exemple simplifié de contexte formel.

TABLE 1 – Un exemple simplifié de contexte formel

Objet	<i>a</i>	<i>b</i>	<i>c</i>
<i>o</i> ₁	1	0	1
<i>o</i> ₂	1	1	0
<i>o</i> ₃	0	1	1

Le même contexte peut être représenté sous forme CSV comme le montre le listing suivant :

; a ; b ; c <i>o</i> 1 ; 1 ; 0 ; 1 <i>o</i> 2 ; 1 ; 1 ; 0 <i>o</i> 3 ; 0 ; 1 ; 1

Des validations sont effectuées avant l'exécution des algorithmes afin de détecter :

- les fichiers manquants,
- les lignes incohérentes,
- les valeurs non binaires,
- les erreurs de parsing.

4.2 Format de sortie des treillis

Les implémentations doivent produire un treillis de concepts au format Graphviz DOT.
 Le format attendu repose sur les conventions suivantes :

- utilisation d'un graphe orienté (**digraph**),
- orientation verticale définie par :

`rankdir=BT`

- un sommet par concept formel,
- une arête par relation de couverture,
- des labels contenant notamment les informations sur les intensions et les extensions.

Les identifiants internes des sommets ne sont pas considérés comme significatifs lors de la comparaison. Deux implémentations peuvent donc utiliser des conventions de nommage différentes tout en représentant exactement le même treillis.

Les labels des concepts suivent généralement une structure du type :

`{id (I: x, E: y)|intension|extension}`

où :

- I correspond à la cardinalité de l'intension,
- E correspond à la cardinalité de l'extension,
- l'intension représente les attributs du concept,
- l'extension représente les objets associés.

La Figure 1 présente un exemple de représentation DOT.

```
digraph G {
    rankdir=BT;
    0 [shape=record,style=filled,fillcolor=lightblue,label="{0 (I: 0, E: 3)||}"];
    1 [shape=record,style=filled,fillcolor=lightblue,label="{1 (I: 1, E: 2)|a\n|}"];
    2 [shape=record,style=filled,label="{2 (I: 2, E: 1)||o2\n}"];
    3 [shape=record,style=filled,fillcolor=lightblue,label="{3 (I: 1, E: 2)|b\n|}"];
    4 [shape=record,style=filled,fillcolor=lightblue,label="{4 (I: 3, E: 0)||}"];
    5 [shape=record,style=filled,label="{5 (I: 2, E: 1)||o1\n}"];
    6 [shape=record,style=filled,label="{6 (I: 2, E: 1)||o3\n}"];
    7 [shape=record,style=filled,fillcolor=lightblue,label="{7 (I: 1, E: 2)|c\n|}"];
    1 -> 0
    2 -> 1
    2 -> 3
    3 -> 0
    4 -> 2
    4 -> 5
    4 -> 6
    5 -> 1
    5 -> 7
    6 -> 3
    6 -> 7
    7 -> 0
}
```

FIGURE 1 – Exemple de la représentation DOT du treillis du tableau 1

4.3 Variantes de représentation

Une difficulté importante provient du fait que plusieurs représentations différentes peuvent décrire un même treillis suivant les options de la commande FCA4J ou suivant l'ordre de génération

des concepts par les différents algorithmes.

Certaines implémentations génèrent notamment :

- des labels complets,
- des labels réduits (*introduced attributes/objects*),
- des ordres de sommets différents,
- des variations syntaxiques dans le DOT.

Ces différences empêchent une comparaison textuelle directe des fichiers produits. Dans notre protocole, une étape de normalisation est donc appliquée afin de convertir les représentations réduites en représentations complètes avant toute comparaison structurelle.

4.4 Définition opérationnelle de la correction

La correction d'une implémentation ne se limite pas au fait que le programme s'exécute sans erreur. Nous distinguons plusieurs niveaux de validité :

- **Correction syntaxique** : le programme produit un fichier DOT valide,
- **Correction fonctionnelle** : le fichier généré représente un graphe exploitable,
- **Correction structurelle** : le treillis généré est équivalent à celui de la référence,
- **Correction théorique** : respect des propriétés fondamentales de l'analyse formelle de concepts [6].

Dans cette étude, le critère principal retenu est la correction structurelle. Une implémentation est considérée comme correcte si le treillis produit est structurellement équivalent au treillis de référence généré par FCA4J 0.4.4.

Cette équivalence repose notamment sur :

- une correspondance bijective entre les ensembles de concepts des deux treillis,
- la préservation des intensions et extensions des concepts associés par cette bijection,
- la préservation de la relation de couverture : deux concepts sont en relation de couverture dans une représentation si et seulement si leurs images par la bijection le sont également dans l'autre,
- en conséquence, un isomorphisme de graphes orientés est établi entre les deux représentations de treillis de concepts.

La comparaison ne dépend donc pas :

- de l'ordre des nœuds,
- des identifiants locaux,
- de la mise en forme textuelle,
- de certaines variations de sérialisation.

4.5 Cas non comparables et statuts d'exécution

Toutes les exécutions ne conduisent pas nécessairement à une comparaison valide.

Plusieurs situations peuvent empêcher l'évaluation structurelle d'un résultat :

- absence de fichier DOT généré,
- timeout,
- erreur d'exécution,
- DOT invalide,
- échec de normalisation.

Afin de distinguer clairement ces situations, le pipeline expérimental attribue un statut explicite à chaque exécution :

- OK,

- FAILED,
- TIMEOUT,
- MISSING_RAW_DOT,
- CONVERSION_FAILED,
- COMPARE_FAILED.

Cette séparation permet de distinguer :

- les implémentations correctes,
- les implémentations incorrectes,
- les résultats non exploitables,
- les cas impossibles à comparer proprement.

5 Méthodologie expérimentale

Cette section décrit le protocole expérimental mis en place afin d'évaluer les implémentations générées par les modèles de langage. L'objectif est de garantir une évaluation reproductible, comparable et indépendante des variations purement syntaxiques des sorties produites.

Le protocole repose sur :

- une implémentation de référence utilisée comme oracle,
- un ensemble de jeux de données synthétiques et réels,
- un pipeline automatisé d'exécution et de comparaison,
- des mesures de performance et de consommation de ressources,
- une procédure de validation structurelle des treillis générés.

5.1 Implémentation de référence

Les comparaisons expérimentales sont réalisées à partir des treillis générés par FCA4J version 0.4.4, utilisée comme implémentation de référence tout au long de cette étude.

Le choix de FCA4J repose principalement sur :

- sa conformité avec les principes classiques de l'Analyse Formelle de Concepts,
- son comportement déterministe,
- sa capacité à calculer les relations de couverture,
- son export des treillis au format Graphviz DOT.

Les treillis produits par FCA4J servent ainsi d'oracle expérimental pour l'évaluation des implémentations générées par les différents LLM. Les détails relatifs :

- aux variations possibles de représentation des treillis,
- à la normalisation des fichiers DOT,
- ainsi qu'à la définition opérationnelle de la correction structurelle

sont présentés respectivement dans la section 4, notamment dans la sous-section dédiée aux variantes de représentation. Dans ce cadre, une implémentation est considérée comme correcte lorsque le treillis qu'elle génère est structurellement équivalent au treillis de référence produit par FCA4J après normalisation.

5.2 Implémentations générées par les LLM

Les implémentations évaluées ont été générées à l'aide de plusieurs modèles de langage distincts. Chaque modèle devait produire une implémentation capable :

- de lire un contexte formel au format CSV,
- de calculer un treillis de concepts,
- de générer une représentation DOT compatible avec le protocole de comparaison.

Deux approches expérimentales ont été étudiées dans le cadre du projet :

- une approche orientée correction fonctionnelle,
- une approche qui demande, en plus, une optimisation de mémoire vive.

Dans la première approche, les modèles étaient principalement encouragés à produire des implémentations correctes capables de générer un treillis valide.

Dans la seconde approche, les prompts mettaient davantage l'accent sur la réduction de la consommation mémoire, notamment par :

- limitation des structures conservées en mémoire,
- partitionnement des traitements,
- externalisation intermédiaire sur disque,
- stratégies de génération progressive des concepts.

Cette distinction permet d'étudier l'impact des contraintes imposées aux modèles sur les propriétés des implémentations produites.

5.3 Choix des modèles de langage

Les modèles évalués dans cette étude ont été sélectionnés afin de couvrir plusieurs catégories représentatives des systèmes de génération de code actuels :

- des modèles propriétaires de pointe,
- des modèles spécialisés ou performants en programmation,
- des modèles open-source récents.

Le choix des modèles s'est principalement appuyé sur les classements publics de la plateforme *Arena*, notamment les catégories liées à la génération de code "Coding leaderboard".

L'objectif n'était pas uniquement d'évaluer les modèles les mieux classés, mais également d'observer :

- la diversité des stratégies algorithmiques produites,
- les différences de robustesse,
- l'impact des contraintes mémoire,
- les écarts entre modèles propriétaires et modèles open-source.

Les modèles suivants ont notamment été utilisés au cours des expérimentations :

- **GPT-5.4** : modèle propriétaire orienté raisonnement et génération de code,
- **GPT-5.3-Codex** : variante spécialisée dans les tâches de programmation,
- **Gemini-3.1** : modèle multimodal de Google classé parmi les meilleurs modèles généralistes récents,
- **Claude Opus 4.6** : modèle particulièrement performant sur les tâches de raisonnement complexe et de génération de code,
- **Kimi-K2** : modèle open-source récent orienté raisonnement et programmation,
- **Qwen-3.6** : modèle open-source utilisé afin d'évaluer le comportement de systèmes plus accessibles et moins coûteux en ressources.

Le choix d'inclure des modèles open-source est également important dans le cadre de cette étude. Contrairement aux modèles propriétaires, ces systèmes peuvent être exécutés localement, adaptés ou réentraînés, ce qui représente un enjeu important pour la reproductibilité et l'accessibilité des outils de génération de code.

Enfin, la diversité des modèles sélectionnés permet d'étudier si certaines propriétés observées, notamment la correction structurelle des treillis ou les stratégies d'optimisation mémoire, sont spécifiques à certains systèmes ou émergent plus généralement dans les implémentations produites par les LLM.

5.4 Jeux de données

Les expériences ont été réalisées sur plusieurs familles de jeux de données représentant différents niveaux de complexité.

5.4.1 Jeux synthétiques

Les datasets synthétiques sont repris des travaux réalisés par le groupe de l'année précédente [5]³. Ils sont principalement utilisés pour étudier :

- le passage à l'échelle,
- l'évolution des temps d'exécution,
- l'évolution de la consommation mémoire,
- le comportement des implémentations face à l'augmentation de la taille des contextes.

Leurs noms sont ainsi codés, indiquant le nombre d'objets et d'attributs :

eg9_9, eg20_20, ..., eg80_80

Ces jeux permettent de contrôler progressivement :

- le nombre d'objets,
- le nombre d'attributs,
- la taille potentielle des treillis générés.

5.4.2 Jeux réels binarisés

Les expériences utilisent également plusieurs datasets réels transformés en contextes binaires, notamment :

- `mushrooms_binarized`,
- `connect-4_binarized`,
- `bank_binarized`,
- `nursery_binarized`,
- `plants_binarized`.

Ces jeux de données permettent d'évaluer les implémentations sur des distributions plus réalistes et potentiellement plus irrégulières que les cas synthétiques.

Tout les datasets réels utilisés appartiennent à Alain Gutierrez⁴ : <https://gite.lirmm.fr/gutierrez/fca4j-benchmark>. Ils ont été construits pour une grande majorité à partir de jeux de données (non binaires) du dépôt UCI⁵.

5.5 Pipeline expérimental

Afin d'automatiser les expériences, un pipeline modulaire a été développé. Celui-ci orchestre l'ensemble des étapes nécessaires à l'exécution et à l'évaluation des implémentations.

Le workflow général est représenté par la chaîne suivante :

exécution → normalisation → comparaison → rapport

3. <https://github.com/OskrJF/TER-M1---Code-and-Test-sets/tree/main/Synthetics>

4. Alain Gutierrez, Ingénieur de recherche CNRS, au LIRMM et développeur de FCA4J.

5. <https://archive.ics.uci.edu/datasets>

5.5.1 Exécution

Chaque implémentation est exécutée indépendamment sur l'ensemble des datasets sélectionnés. Le pipeline :

- prépare les chemins d'entrée et de sortie,
- applique les limites de temps,
- capture les erreurs d'exécution,
- collecte les métriques système,
- archive les artefacts produits.

5.5.2 Normalisation

Comme dit précédemment, certaines implémentations produisent des treillis utilisant des labels réduits (inférence du contenu des noeuds).

Une étape de normalisation est donc appliquée afin de :

- reconstruire les intensions complètes,
- reconstruire les extensions complètes,
- uniformiser la représentation des concepts,
- produire un format comparable avec la référence.

Cette étape est essentielle afin d'éviter les faux négatifs liés uniquement à des différences de représentation.

5.5.3 Comparaison structurelle

La comparaison structurelle définie dans la section 4.4 est appliquée sur les formes normalisées.

5.5.4 Génération des rapports

À l'issue des comparaisons, le pipeline génère automatiquement :

- des rapports Markdown,
- des tableaux de résultats,
- des logs d'exécution,
- des résumés de statuts,
- des métriques de ressources.

Chaque exécution est associée à un identifiant unique (*run id*) permettant l'archivage et la reproduction des expériences.

5.6 Mesures de performance et de ressources

Le protocole expérimental mesure plusieurs indicateurs afin d'évaluer les propriétés pratiques des implémentations générées. Nous les présentons dans cette section.

5.6.1 Temps d'exécution

Le type de mesure temporelle utilisé est le temps CPU (s). Les mesures sont collectées automatiquement via des wrappers d'exécution spécifiques.

5.6.2 Wrappers de mesure des ressources

Afin d'évaluer précisément l'empreinte des différentes implémentations, nous avons développé un wrapper d'instrumentation dédié à la mesure des ressources, sans modification du code des algorithmes testés. Pendant l'exécution de chaque algorithme, un processus de surveillance associé

(`resource_monitor.py`) échantillonne périodiquement les ressources système du processus racine ainsi que de ses processus enfants. Cette approche permet de capturer une estimation globale de l'exécution, y compris lorsque des sous-processus sont utilisés. Ces mesures sont agrégées sous forme de pics observés durant toute la durée de l'exécution, fournissant ainsi une estimation conservative de l'utilisation des ressources. L'ensemble du dispositif repose sur une lecture des informations exposées par le système via `/proc`, ce qui permet une instrumentation légère et non intrusive, mais implique une dépendance à un environnement de type Linux ou WSL. Cette approche garantit néanmoins une comparaison homogène entre les différentes implémentations évaluées, indépendamment de leur langage ou de leur structure interne.

5.6.3 Consommation mémoire

La mémoire maximale utilisée (*RSS max en MiB*) est surveillée pendant l'exécution des algorithmes. Cette mesure est particulièrement importante dans le cadre des implémentations *memory-aware*, dont l'objectif principal est de limiter la consommation mémoire lors du calcul des treillis.

5.6.4 Utilisation disque et I/O

Certaines implémentations utilisent des mécanismes d'externalisation partielle des données. Le pipeline mesure donc également :

- les écritures disque,
- la taille maximale des répertoires temporaires,
- certaines statistiques d'entrées/sorties système.

Les mesures sont réalisées sous Linux à partir des informations disponibles dans `/proc`, sans dépendances externes supplémentaires.

5.7 Reproductibilité

Une attention particulière a été portée à la reproductibilité des expériences. Les éléments suivants sont archivés pour chaque exécution :

- configurations JSON,
- paramètres du pipeline,
- versions des implémentations,
- logs complets,
- sorties intermédiaires,
- rapports générés.

Les expériences ont été réalisées dans un environnement Linux contrôlé avec :

- une version fixe de FCA4J,
- des versions déterminées des interpréteurs utilisés,
- des limites de temps identiques entre les implémentations.

Cette organisation permet de relancer les expériences et de reproduire les résultats obtenus dans des conditions similaires.

5.7.1 Modèles de langage utilisés

Les implémentations évaluées dans cette étude ont été générées à l'aide de plusieurs modèles de langage de grande taille (LLM). Les générations ont été réalisées à différentes dates afin de prendre en compte les versions disponibles au moment des expérimentations.

Les modèles utilisés sont les suivants :

- Claude Opus 4.6 — 12 mars 2026,
- Claude Sonnet 4.6 — 19 mars 2026,
- GPT-5.3 Codex (*xhigh*) — 19 mars 2026,
- GPT-5.4 (*medium*) — 10 mai 2026,
- Qwen 3.6 (*thinking*) — 9 mai 2026,
- Kimi K2.5 (*thinking*) — 9 mai 2026.

Les termes *xhigh*, *medium* et *thinking* correspondent aux modes de raisonnement ou niveaux d'effort de génération proposés par les plateformes utilisées au moment des expériences. De plus les LLM Claude Opus 4.6, Claude Sonnet 4.6 et GPT-5.3 Codex ont été utilisés par le biais de Github Copilot, tandis que GPT-5.4, Qwen 3.6 et Kimi K2.5 ont été utilisés depuis leurs sites dédiés.

5.7.2 Analyse du prompt utilisé dans les implémentations sans contrainte mémoire

Pour la construction de nos prompt nous avons suivi les recommandations de :

- Comment rédiger des prompts efficaces, Mammouth ai [12],
- Guide de prompt engineering, Openai [13],
- Prompt engineering overview, Anthropic [1],
- Prompt Engineering Guide [14].

Le prompt utilisé pour les implémentations sans contrainte mémoire, en Annexe A.1.1, joue un rôle central dans la qualité des résultats obtenus, notamment en ce qui concerne la cohérence structurelle des treillis générés et leur équivalence avec FCA4J.

Structure générale du prompt

Le prompt est fortement structuré et impose explicitement :

- un rôle précis (*expert en algorithmique et FCA*),
- un cadre théorique rappelant les notions fondamentales (concept formel, extension, intension, treillis de concepts),
- une tâche algorithmique complète et non ambiguë,
- des contraintes strictes sur les formats d'entrée et de sortie,
- une exigence de programme autonome en C++17.

Cette structuration réduit fortement l'ambiguïté sémantique et limite les interprétations divergentes du problème.

Qualités principales du prompt

1. Spécification formelle du problème

Le prompt décrit précisément :

- le format CSV attendu,
- la construction des concepts formels,
- la définition du treillis et du diagramme de Hasse,
- les règles de génération des identifiants et des labels.

Cette formalisation réduit les degrés de liberté laissés au modèle, ce qui favorise la reproductibilité.

Les stratégies utilisées dans ce prompt sont :

- Role conditioning,
- Task decomposition,
- Structured prompting.

2. Contrôle strict de la sortie

Le format `.dot` est entièrement contraint (structure, couleurs, labels, arêtes), ce qui force une conformité syntaxique élevée et limite les erreurs de génération de code ou de représentation.

3. Encadrement algorithmique

Le prompt impose explicitement :

- une génération complète du treillis en une seule passe,
- l'utilisation d'un algorithme FCA standard (Next Closure ou équivalent),
- l'interdiction de dépendances externes.

Cela oriente fortement le modèle vers une solution algorithmique déterministe plutôt que heuristique.

4. Réduction de l'ambiguïté conceptuelle

La redéfinition explicite des notions FCA (extension, intension, générateurs) limite les erreurs d'interprétation fréquentes dans les LLM sur les concepts mathématiques structurés.

Utilisation du prompt

Ce prompt a été soumis une seule fois par modèle dans ce protocole, afin de garantir une comparaison homogène entre les implémentations générées.

5.7.3 Analyse du prompt utilisé dans les implémentations avec contrainte mémoire

Le prompt utilisé dans la version avec contrainte mémoire, en Annexe A.1.2, introduit une modification fondamentale de paradigme : il ne s'agit plus uniquement de produire un algorithme correct pour construire un treillis de concepts, mais de concevoir une implémentation capable de fonctionner sous "contrainte forte de mémoire vive", en intégrant des mécanismes explicites de partitionnement et de persistance disque.

Structure générale du prompt

Le prompt est structuré de manière extrêmement directive et décomposé en étapes algorithmiques explicites

- définition du rôle (expert FCA et optimisation mémoire),
- rappel du contexte mathématique,
- objectif fonctionnel multi-étapes,
- contraintes générales fortes,
- stratégie mémoire imposée,
- décomposition algorithmique en 10 étapes,
- spécification stricte des fonctions à implémenter,
- format de sortie attendu avec structure figée.

Cette granularité impose une architecture logicielle plutôt qu'un simple algorithme. Les stratégies utilisées dans ce prompt sont :

- Role conditioning,
- Structured prompting,
- Chain-of-Thought,
- Step-by-step decomposition,
- Divide-and-conquer.

Qualités principales du prompt

1. Transformation du problème en problème système

Contrairement au prompt sans contrainte mémoire, ici le problème n'est plus uniquement algorithmique mais devient un problème de système de traitement de données :

- gestion de partitions,
- écriture sur disque,
- recomposition finale,
- libération explicite de mémoire.

Cela force le modèle à raisonner en termes de pipeline et non uniquement de structure mathématique.

2. Introduction explicite de la contrainte mémoire

Une différence majeure réside dans l'introduction explicite de contraintes d'exécution :

- interdiction de stocker le treillis complet en mémoire,
- obligation de libération entre partitions,
- écriture intermédiaire sur disque.

Cette contrainte induit une optimisation implicite de la complexité mémoire, absente du prompt non contraint.

3. Décomposition algorithmique structurée

Le prompt impose une décomposition fine en étapes (1 à 10), couvrant :

- chargement des données,
- opérateur de fermeture,
- énumération des concepts,
- partitionnement,
- gestion disque,
- recomposition,
- calcul des arêtes,
- génération DOT.

Cette structure réduit fortement la liberté d'implémentation mais augmente la cohérence globale du pipeline.

4. Spécification logicielle complète

Le prompt impose une architecture logicielle complète avec :

- fonctions obligatoires (`load_context`, `closure`, etc.),
- séparation claire des responsabilités,
- exigences de modularité et robustesse,
- format de sortie strict.

Cela rapproche la tâche d'un cahier des charges industriel.

Limites structurelles du prompt

Ce prompt introduit également plusieurs sources de fragilité :

- une augmentation de la complexité contextuelle due à la décomposition en 10 étapes détaillées,
- difficulté à maintenir la cohérence globale entre partitions,
- risque élevé de duplication ou d'omission de concepts lors de la recomposition,
- complexité accrue de la génération du diagramme de Hasse,
- forte sensibilité aux erreurs de sérialisation et de fusion.

Utilisation du prompt

Dans le cadre de la version avec contraintes mémoire, chaque prompt a été soumis trois fois par modèle afin de réduire l'impact du non-déterminisme des modèles et d'obtenir une estimation plus robuste des comportements observés.

5.8 Cartographie et organisation du dépôt

Afin d'assurer la reproductibilité des expériences et la traçabilité des résultats, l'ensemble du code, des jeux de données et des sorties expérimentales est organisé selon une structure de dépôt Git clairement définie.

Le dépôt est structuré en plusieurs répertoires principaux :

- **Par_Algo_direct** : contient les implémentations sans contrainte mémoire (*memory-unaware*), incluant les algorithmes générés par les différents LLM ainsi que les sorties `.dot` associées,
- **Par_recomposition** : regroupe les implémentations avec contrainte mémoire (*memory-aware*), basées sur une approche par partitionnement et recomposition,
- **Dataset** : contient l'ensemble des jeux de données utilisés pour les expérimentations,
- **Resultats** : regroupe les résultats expérimentaux sous forme de rapports et de fichiers `.md`, incluant notamment les sorties de pipelines et les tableaux de benchmark.

Le répertoire **Par_Algo_direct** est structuré comme suit :

- **Benchmark** : contient les résultats de benchmark (également disponibles dans **Resultats**),
- **FCA4J** : inclut l'archive de l'implémentation de référence FCA4J,
- **Large Language Model** : regroupe les sous-répertoires correspondant à chaque LLM testé, contenant à la fois les algorithmes générés et les fichiers `.dot` produits,
- **Outils** : contient les scripts utilitaires (e.g. `compare_lattices.py`, `induced_to_full_dot.py`, `run_benchmark.sh`),
- **Prompt** : regroupe les prompts utilisés pour la génération des implémentations.

Le répertoire **Par_recomposition** suit une organisation similaire, avec quelques spécificités :

- **FCA4J** : implémentation de référence,
- **«LLM»** : dossiers par modèle testé, contenant les algorithmes générés et les sorties `.dot`,
- **Outils** : scripts de traitement et d'évaluation (e.g. `compare_lattices.py`, `induced_to_full_dot.py`, `run_pipeline.py`), les résultats détaillés des exécutions de pipeline sont stockés dans `Outils/outputs/<id_run>/report/rapport_pipeline.md`, accompagnés des métadonnées associées dans `Outils/outputs/<id_run>/metadata`.
- **Prompt** : regroupe les prompts utilisés pour la génération des implémentations.

Cette organisation permet d'assurer une séparation claire entre les deux paradigmes étudiés (génération directe et approche mémoire contrainte), tout en facilitant la reproductibilité des expériences et l'analyse comparative des résultats.

5.9 Environnement matériel et logiciel

L'ensemble des expériences a été réalisé sur une machine unique afin de garantir des conditions d'exécution homogènes entre les différentes implémentations évaluées.

5.9.1 Système d'exploitation

Les expérimentations ont été effectuées sous :

- Ubuntu 22.04.5 LTS.

5.9.2 Configuration matérielle

La machine utilisée possède les caractéristiques suivantes :

- **Processeur (CPU)** : Intel Core i5-13600H (13^e génération), 12 cœurs, 16 threads, fréquence maximale de 4.8 GHz,
- **Mémoire vive (RAM)** : 16 GiB DDR5 SODIMM à 5600 MHz,
- **Stockage** : SSD NVMe Samsung PM9B1 de 512 GB,
- **GPU** : NVIDIA dédiée et Intel Iris Xe intégrée,
- **Machine** : Dell Precision 3581 (*notebook*).

Les calculs étudiés dans ce travail reposent principalement sur le processeur et la mémoire vive. Le GPU n'est pas utilisé pour l'exécution des algorithmes FCA évalués.

La description précise de l'environnement matériel est importante dans le cadre de cette étude, les performances observées étant fortement dépendantes :

- des capacités mémoire disponibles,
- des performances CPU,
- des vitesses d'entrées/sorties disque.

Cette configuration constitue également une contrainte réaliste pour l'étude des implémentations *memory-aware*, l'objectif étant précisément de limiter les dépassements de mémoire sur des machines disposant de ressources limitées.

6 Implémentations générées

Cette section présente les implémentations produites par les modèles de langage évalués dans le cadre du projet. Deux objectifs distincts avaient été définis :

- une implémentation classique visant principalement la correction fonctionnelle et la génération complète du treillis de concepts,
- une implémentation orientée efficacité mémoire, destinée à limiter l'utilisation de la RAM lors du calcul des concepts formels et de la génération du diagramme de Hasse.

Parmi l'ensemble des implémentations obtenues, nous avons choisi d'analyser plus en détail les deux suivantes :

- `lattice.cpp` de Claude Sonnet 4.6 pour la partie sans contrainte mémoire,
- `lattice2.py` de Claude Opus 4.6 pour la partie d'efficacité mémoire.

Les deux approches reposent sur l'algorithme Next Closure de Ganter pour l'énumération des ensembles fermés, mais diffèrent fortement dans leurs structures de données, leurs stratégies de stockage et leurs compromis algorithmiques.

6.1 Implémentation classique sans contrainte mémoire

L'implémentation générée pour cette partie du projet a été produite en C++. L'objectif principal était ici d'obtenir une implémentation fonctionnelle complète sans contrainte particulière sur l'utilisation mémoire.

6.1.1 Structure générale

Le programme :

1. charge un contexte formel depuis un fichier CSV,
2. calcule tous les concepts formels via Next Closure,

3. construit le diagramme de Hasse,
4. exporte le résultat au format DOT.

L'ensemble des concepts est conservé en mémoire pendant toute l'exécution.

6.1.2 Structures de données

Le contexte formel est représenté par :

- une liste d'objets,
- une liste d'attributs,
- une matrice booléenne objet $\times$ attribut.

Chaque concept est représenté par :

- un extension,
- une intension,

eux-mêmes stockés sous forme de vecteurs booléens.

6.1.3 Implémentation de Next Closure

L'algorithme Next Closure a été retenu par le modèle parmi les différentes approches proposées (Next Closure, Close by One ou équivalents) pour énumérer les ensembles fermés. Le programme :

- construit un ensemble d'attributs possible,
- applique l'opérateur de fermeture,
- vérifie la condition d'ordre définissant la génération des ensembles,
- produit le concept suivant.

Cette implémentation suit fidèlement la formulation théorique classique de l'algorithme de Ganter.

6.1.4 Calcul du diagramme de Hasse

Le modèle à choisi de calculer les relations de couverture via une approche quadratique :

- comparaison de tous les couples de concepts,
- vérification des inclusions d'intensions,
- suppression des relations transitives.

Cette méthode est correcte mais devient rapidement coûteuse lorsque le nombre de concepts augmente fortement.

6.1.5 Export Graphviz

Le résultat final est exporté sous forme de fichier Graphviz DOT.

Chaque concept contient :

- les attributs introduits par chaque concept,
- les objets introduits,
- des couleurs spécifiques selon certaines propriétés des concepts.

Le treillis obtenu peut ensuite être visualisé avec Graphviz.

6.1.6 Influence du prompt sur l'implémentation

Impact sur les résultats expérimentaux

La qualité du prompt explique en grande partie les performances observées des modèles, Claude Sonnet 4.6 et Gemini 3.1 Pro à produire des résultats systématiquement équivalents à FCA4J.

Limites observées

Ces limitations ont été observées empiriquement à partir d'une exécution unique du prompt pour chacun des modèles testés. L'analyse repose donc sur une comparaison ponctuelle des sorties obtenues.

Malgré sa précision, ce prompt présente plusieurs limitations :

- une forte dépendance à la capacité du modèle à implémenter correctement un algorithme complexe en une seule passe,
- une sensibilité aux erreurs de format (DOT invalide, incohérences d'identifiants),
- une absence de gestion explicite de la complexité mémoire et temporelle,
- une difficulté pour les modèles moins performants à respecter simultanément toutes les contraintes.

Conclusion

Ce prompt illustre un cas de *prompting algorithmique contraint*, où la performance ne dépend pas uniquement du modèle, mais également de la précision des spécifications fournies. Il constitue un facteur déterminant dans l'obtention de treillis structurellement corrects dans les expérimentations réalisées.

6.1.7 Analyse de l'approche

Cette implémentation met en évidence qu'un LLM peut produire une implémentation FCA complète relativement proche des implémentations académiques classiques. Plusieurs éléments positifs ont été observés :

- structure du code claire,
- séparation correcte des responsabilités,
- commentaires cohérents,
- bonne compréhension de l'algorithme Next Closure,
- export DOT directement exploitable.

6.2 Implémentation orientée efficacité mémoire

L'implémentation générée pour la partie "économie mémoire" a été produite en Python. Son objectif principal était de réduire l'empreinte mémoire lors du calcul des concepts formels, notamment pour les contextes contenant un grand nombre de concepts.

6.2.1 Principes généraux

Plusieurs choix de conception, issus à la fois des contraintes explicites du prompt et des décisions d'implémentation prises par les modèles, ont été réalisés afin de limiter l'utilisation de la mémoire :

- représentation des ensembles d'objets et d'attributs sous forme de bitmasks,
- partitionnement des concepts générés,
- écriture intermédiaire sur disque,
- rechargement progressif des partitions,
- calcul adaptatif de la taille des lots (`batch size`) selon la mémoire disponible,
- libération explicite des structures temporaires via le ramasse-miettes (`gc.collect()`).

L'algorithme conserve uniquement le contexte formel en mémoire tandis que les concepts déjà calculés sont sérialisés sur disque au fur et à mesure de l'exécution.

6.2.2 Représentation par bitmasks

La représentation des ensembles d'objets et d'attributs sous forme de bitmasks n'est pas explicitement imposée par le prompt, mais apparaît dans certaines implémentations comme une optimisation visant à répondre aux contraintes de performance et de mémoire.

Les ensembles d'objets et d'attributs sont encodés sous forme d'entiers binaires. Chaque bit représente la présence ou l'absence d'un élément dans un ensemble.

Cette représentation permet :

- des opérations d'intersection rapides via des opérations bit à bit,
- une réduction importante de l'empreinte mémoire,
- une amélioration des performances des opérateurs de dérivation.

Les opérations de fermeture reposent alors essentiellement sur des opérations AND entre entiers. Cette représentation est plus complexe à manipuler mais beaucoup moins coûteuse en mémoire que l'approche par bitmasks.

6.2.3 Partitionnement des concepts

L'énumération des concepts via Next Closure produit un flux de concepts ordonné lexicographiquement. Plutôt que de conserver l'ensemble des concepts en mémoire, ceux-ci sont regroupés par lots puis sauvegardés sur disque. Chaque partition contient un nombre limité de concepts et peut elle-même être découpée en plusieurs fichiers JSON (chunks).

Cette stratégie permet :

- de borner l'utilisation mémoire,
- de traiter des contextes plus volumineux,
- d'éviter la saturation mémoire observée dans certaines implémentations FCA classiques.

6.2.4 Calcul adaptatif de la taille des lots

La taille des partitions est calculée dynamiquement en fonction de la mémoire disponible sur le système.

L'implémentation estime la quantité de RAM libre à partir du fichier Linux `/proc/meminfo` puis ajuste automatiquement le nombre de concepts conservés simultanément en mémoire.

Cette stratégie permet :

- une meilleure adaptation aux ressources matérielles disponibles,
- une réduction du risque d'épuisement mémoire,
- une amélioration de la robustesse sur des machines disposant de capacités différentes.

6.2.5 Calcul de la relation de couverture

Après rechargement des partitions, les concepts sont fusionnés puis triés de manière déterministe.

Le calcul du diagramme de Hasse repose sur :

- un index par niveaux basé sur la cardinalité des intensions,
- une élimination des relations transitives,
- une recherche gloutonne des couvertures directes.

Cette optimisation limite le nombre de comparaisons inutiles entre concepts.

6.2.6 Génération du fichier DOT

Comme pour l’approche précédente, le treillis est exporté au format DOT.

Le programme calcule également :

- la taille de l’intension,
- la taille de l’extension,
- les attributs propres,
- les objets propres.

6.2.7 Influence du prompt sur l’implémentations

Impact sur les résultats expérimentaux

Les résultats observés montrent que la structuration du prompt a un effet ambivalent :

- certains modèles (notamment les plus performants) parviennent à respecter la logique globale de partitionnement tout en maintenant la correction structurelle sur des cas simples,
- cependant, la complexité cumulative des contraintes introduit de nombreuses erreurs de conversion ou d’incohérence structurelle,
- les performances se dégradent fortement sur les jeux de données de grande taille.

On observe ainsi que la contrainte mémoire améliore la scalabilité théorique mais dégrade la fiabilité pratique des implémentations générées.

Conclusion

Ce prompt illustre une transition d’un *prompt algorithmique contraint* vers un *prompt de conception de système distribué à mémoire contrainte*. Si cette approche permet théoriquement une meilleure scalabilité, elle introduit une complexité structurelle significative qui explique les erreurs observées dans les résultats expérimentaux, notamment sur les jeux de données de grande taille.

6.2.8 Analyse de l’approche

Cette implémentation montre qu’un LLM est capable de générer :

- une architecture logicielle complexe,
- des optimisations mémoire explicites,
- une gestion avancée du stockage disque,
- des mécanismes de partitionnement cohérents,
- une documentation technique particulièrement détaillée.

Cependant, la complexité globale demeure fortement dépendante du nombre de concepts formels, qui peut rester exponentiel dans le pire cas.

6.3 Comparaison des deux approches

Les deux implémentations reposent sur les mêmes fondements théoriques mais diffèrent fortement dans leurs objectifs et leurs compromis.

Ces résultats montrent que les modèles de langage sont capables de produire des implémentations significativement différentes à partir d’objectifs algorithmiques distincts. Ils illustrent également la capacité des LLM à intégrer des contraintes non fonctionnelles complexes, comme la limitation de l’usage mémoire.

Critère	Implémentation mémoire	Implémentation classique
Langage	Python	C++
Objectif principal	Réduction RAM	Simplicité fonctionnelle
Représentation	Bitmasks	Vecteurs booléens
Stockage des concepts	Disque + partitions	Mémoire vive
Gestion mémoire	Adaptative	Aucune optimisation spécifique
Complexité pratique	Plus scalable	Plus rapide
Diagramme de Hasse	Optimisé par niveaux	Comparaison quadratique
Lisibilité	Très documentée	Plus concise

TABLE 2 – Comparaison des implémentations générées

7 Résultats expérimentaux

Cette section présente les résultats obtenus lors de l’évaluation des différentes implémentations générées pour la construction de treillis de concepts. Les expériences ont été réalisées sur plusieurs familles de jeux de données, incluant des contextes formels classiques binarisés ainsi que des instances synthétiques.

Les évaluations portent sur plusieurs dimensions :

- la correction structurelle des treillis générés,
- les performances temporelles,
- la consommation mémoire,
- les volumes d’écriture disque,
- la robustesse face aux jeux de données complexes.

7.1 Résultats de correction structurelle

Le principal critère d’évaluation retenu est l’équivalence structurelle avec les treillis générés par FCA4J. Une implémentation est considérée comme correcte lorsque le treillis produit est isomorphe au treillis de référence après normalisation des fichiers DOT. Les résultats montrent des différences importantes entre les implémentations évaluées.

7.1.1 Implémentations sans contrainte mémoire

Le Tableau 3 synthétise les résultats de correction structurelle obtenus pour les principales implémentations évaluées. Ces résultats proviennent de cinq exécutions indépendantes (runs) de chaque modèle, permettant d’observer la stabilité des comportements générés. L’ensemble des sorties expérimentales est disponible dans le dossier **resultats** du dépôt Git.

Implémentation	Taux d’équivalence avec FCA4J
Claude Sonnet 4.6	100%
Gemini 3.1 Pro (preview)	100%
GPT-5.3 Codex	31.25%
Qwen 3.6	0%

TABLE 3 – Taux d’équivalence structurelle avec FCA4J

Les implémentations générées par Claude Sonnet 4.6 et Gemini 3.1 Pro (preview) produisent systématiquement des treillis structurellement équivalents à ceux de FCA4J sur l’ensemble des jeux de données évalués. À l’inverse, GPT-5.3 Codex échoue sur la majorité des jeux de données réels et ne produit des résultats corrects que sur les jeux de données synthétiques. L’implémentation générée par Qwen ne produit aucun treillis structurellement équivalent.

7.1.2 Implémentations orientées économie mémoire

Dans le cadre des implémentations optimisées pour réduire l’usage mémoire, les résultats de correction structurelle restent très variables.

Implémentation	Équivalence avec FCA4J
Claude Opus 4.6	100%
GPT-5.3 Codex	Correct uniquement sur les jeux synthétiques
GPT-5.4	
Qwen 3.6	
kimi k2.5	
	0%

TABLE 4 – Résultats d’équivalence des implémentations optimisées mémoire

L’implémentation de **Claude Opus 4.6** est la seule à conserver une équivalence structurelle complète tout en réduisant fortement la consommation mémoire par rapport à FCA4J.

7.2 Résultats de performance

Les performances temporelles montrent des écarts importants entre les implémentations.

7.2.1 Temps d’exécution

FCA4J reste globalement l’implémentation la plus rapide sur les grands jeux de données. Les implémentations générées par Claude Sonnet 4.6 obtiennent également de bonnes performances tout en conservant une correction structurelle complète. À l’inverse, certaines implémentations générées présentent des temps d’exécution très élevés, voire des timeouts déclenché 3600s (CPU) après le début du processus. Le Tableau 5 compare les performances des approches classiques et des approches optimisées mémoire sur les datasets synthétiques les plus coûteux.

Implémentation	Stratégie	eg40_40	eg50_50	Équivalence
FCA4J	Référence	1.26 s	5.89 s	Oui
Claude Sonnet 4.6	Classique	40.72 s	13 min 18 s	Oui
Claude Opus 4.6	Memory-aware	9.55 s	180.72 s	Oui
GPT-5.3 Codex	Classique	48.02 s	Timeout	Oui partiel
GPT-5.3 Codex	Memory-aware	188.63 s	Timeout	Oui partiel
Gemini 3.1 Pro (preview)	Classique	4 min 32 s	Timeout	Oui

TABLE 5 – Comparaison des stratégies classiques et memory-aware

Les résultats montrent que les stratégies d’économie mémoire influencent fortement les performances :

- certaines implémentations, comme **Claude Opus 4.6**, améliorent l’usage mémoire en gardant un temps d’exécution correct,
- d’autres, comme **GPT-5.3 Codex**, réduisent la mémoire mais augmentent fortement le coût temporel.

7.2.2 Analyse par famille de datasets

Les jeux synthétiques mettent principalement en évidence les limites de passage à l’échelle des algorithmes.

Les datasets réels binarisés révèlent quant à eux des différences importantes de robustesse :

- certains algorithmes échouent rapidement,

- d’autres produisent des résultats corrects mais avec une forte consommation de temps et mémoire,
- plusieurs implémentations génèrent des treillis incorrects avec des temps d’exécution très faibles.

7.3 Scalabilité

Les expériences réalisées sur les datasets synthétiques montrent une croissance très rapide des coûts de calcul lorsque la taille des treillis augmente. Les implémentations incorrectes présentent généralement une augmentation importante des écritures disque intermédiaires.

7.4 Consommation des ressources

7.4.1 Consommation mémoire

Les différences de pics de consommation mémoire sont particulièrement importantes entre les implémentations.

Implémentation	RAM maximale observée	Dataset
FCA4J	845.99 MiB	eg50_50
Claude Opus 4.6	107.52 MiB	connect-4_binarized
GPT-5.3 Codex	34.00 MiB	eg50_50
GPT-5.4	129.96 MiB	connect-4_binarized
Gemini 3.1 Pro (preview)	1.42 GiB	plants_binarized

TABLE 6 – Consommation mémoire maximale

L’implémentation **Claude Opus 4.6** et **GPT-5.3 Codex** réduisent fortement l’usage mémoire par rapport à **FCA4J** tout en conservant une correction structurelle complète. À l’inverse, certaines implémentations comme **Gemini 3.1 Pro (preview)** présentent des consommations mémoire très importantes ainsi que des volumes d’écriture disque particulièrement élevés.

7.4.2 Écritures disque

Les stratégies de partitionnement influencent fortement les volumes d’écriture disque. Certaines implémentations génèrent plusieurs gigaoctets de données temporaires même sur des jeux de données de taille moyenne.

Par exemple :

- **Gemini 3.1 Pro (preview)** atteint jusqu’à 10.79 GiB d’écritures disque,
- **GPT-5.3 Codex** atteint 2.90 GiB,
- **Claude Opus 4.6** reste limité à 34.25 MiB.

Ces résultats montrent que les stratégies de gestion mémoire influencent également fortement les coûts d’I/O.

7.5 Robustesse

Les implémentations les plus robustes sont celles générées par **Claude Sonnet 4.6**, **Claude Opus 4.6** ainsi que **Gemini 3.1 Pro (preview)** (dans l’approche classique), qui conservent une correction structurelle complète sur l’ensemble des jeux de données évalués. Les implémentations de **GPT-5.3 Codex** restent aussi robuste sur les jeux de données synthétiques, mais montre un haut taux d’échec sur les jeux de données réels. Les autres implémentations tendent au contraire à produire des approximations incorrectes du treillis ou des structures incomplètes.

8 Analyse des erreurs et discussion

Cette section est basée sur les analyses d’erreurs issues du fichier `Outils/outputs/20260510-201419/report/rapport_pipeline.md`, également archivé dans le dépôt sous `resultats/rapport_pipeline_memory_aware.md`. Elle analyse les écarts de correction observés entre les différentes implémentations, en mettant en évidence les principales sources d’échec, les comportements émergents des modèles, ainsi que les compromis induits par les choix algorithmiques et les contraintes d’exécution.

8.1 Interprétation générale des résultats

Les résultats expérimentaux montrent une séparation nette entre trois catégories d’implémentations :

- des implémentations structurellement robustes et cohérentes avec FCA4J,
- des implémentations partiellement correctes mais instables sur les cas réels (GPT-5.3 Codex),
- des implémentations structurellement incorrectes ou non fonctionnelles (dans les 2 parties : Qwen-3.6 et dans la partie avec restrictions de mémoire : kimi k2.5 et Gemini 3.1 Pro (preview)).

Cette hiérarchie suggère que la correction du treillis ne dépend pas uniquement de la capacité algorithmique brute, mais également de la stabilité des représentations intermédiaires et de la gestion des contraintes de sortie.

8.2 Causes principales d’échec

L’analyse des logs de pipeline permet d’identifier plusieurs familles d’erreurs récurrentes.

Erreurs de parsing du CSV

Une première source d’erreur provient de l’interprétation du format d’entrée :

- ambiguïtés sur les séparateurs,
- gestion incorrecte des en-têtes,
- désalignement entre objets et attributs.

Ces erreurs se propagent directement vers les étapes de fermeture et faussent l’ensemble du treillis.

Génération DOT non conforme

Une part importante des échecs provient de fichiers DOT invalides :

- nœuds mal formés ou incomplets,
- incohérences d’identifiants,
- arêtes manquantes ou structure non valide,
- non-respect des contraintes de label et de style.

Ces erreurs empêchent la validation structurelle du graphe en aval.

Erreurs algorithmiques sur les concepts

Certaines implémentations produisent des concepts incorrects :

- violations de la propriété de fermeture (intent/extent incohérents),
- doublons de concepts,
- concepts partiellement calculés ou incomplets.

Ces erreurs indiquent une instabilité dans l’opérateur de fermeture ou dans les stratégies d’énumération.

Incohérences structurelles du treillis

Dans certains cas, la structure globale du treillis est invalide :

- absence de sommet ou de fond unique,
- violation de l'ordre partiel,
- relations de couverture incorrectes ou incomplètes.

Ces problèmes traduisent des erreurs dans la construction du diagramme de Hasse ou dans la filtration des concepts intermédiaires.

8.3 Comportements émergents des implémentations

Les résultats mettent en évidence plusieurs tendances comportementales :

- les modèles les plus performants privilégient une exhaustivité correcte au prix d'un coût plus élevé,
- certains modèles introduisent des heuristiques implicites de simplification,
- les modèles moins robustes échouent dès l'augmentation de la complexité structurelle,
- certaines sorties restent syntaxiquement valides mais structurellement incohérentes.

8.4 Impact indirect des contraintes de prompt

Bien que les prompts aient été analysés précédemment, plusieurs effets indirects expliquent les erreurs observées :

- surcharge de contraintes simultanées (format, algorithme, mémoire, structure),
- conflits implicites entre exhaustivité et contraintes de performance,
- difficulté à maintenir une cohérence globale sur plusieurs étapes,
- rigidité excessive des spécifications réduisant la flexibilité de correction des modèles.

8.5 Compromis algorithmiques

Les implémentations étudiées illustrent plusieurs compromis fondamentaux :

Vitesse vs exhaustivité

Les approches rapides tendent à simplifier le calcul des concepts, au risque de perdre en complétude.

Mémoire vs cohérence globale

Les stratégies de partitionnement réduisent la mémoire utilisée mais augmentent le risque de désalignement entre partitions.

Robustesse vs complexité de sortie

La génération de sorties riches (DOT structuré, labels, couleurs) augmente significativement le risque d'erreurs de format.

8.6 Synthèse

L'ensemble des résultats montre que les erreurs ne proviennent pas uniquement d'une faiblesse algorithmique des modèles, mais d'une interaction complexe entre :

- la complexité intrinsèque de l'Analyse Formelle de Concepts,
- la rigidité des formats de sortie imposés,
- les contraintes de mémoire et de structure,
- et la capacité des modèles à maintenir une cohérence multi-étapes.

Cette interaction explique les différences de performance observées entre implémentations et met en évidence les limites actuelles des LLM pour la génération fiable d'algorithmes complexes à grande échelle.

9 Menaces à la validité (Threats to validity)

Cette section discute les principales menaces à la validité des résultats obtenus dans cette étude. L'objectif est d'identifier les facteurs susceptibles d'affecter l'interprétation des performances observées, ainsi que la généralisation des conclusions.

9.1 Validité interne

La validité interne concerne la fiabilité des résultats obtenus dans le cadre expérimental défini. Plusieurs sources de biais ou d'instabilité peuvent être identifiées :

Instrumentation et mesures

Les métriques utilisées (temps d'exécution, consommation mémoire, I/O disque, succès/échec) reposent sur des outils d'instrumentation pouvant introduire des variations :

- précision limitée des compteurs mémoire et I/O,
- surcharge induite par la mesure elle-même.

Timeouts et interruptions d'exécution

L'utilisation de timeouts pour limiter les exécutions longues introduit un biais structurel :

- certaines exécutions sont classées comme *TIMEOUT* alors qu'elles auraient pu aboutir,
- la frontière entre échec algorithmique et limitation arbitraire du temps n'est pas toujours claire,
- les datasets les plus complexes sont mécaniquement pénalisés.

Non-déterminisme potentiel

Certaines implémentations peuvent introduire du non-déterminisme :

- ordre d'itération sur structures de données,
- gestion de parallélisme implicite ou optimisations internes,
- variations dans les allocations mémoire.

Cela peut affecter la reproductibilité exacte de certains résultats.

9.2 Validité externe

La validité externe concerne la capacité des résultats à être généralisés à d'autres contextes.

Généralisation aux datasets

Les datasets utilisés couvrent un ensemble varié de contextes, mais restent limités :

- prédominance de contextes binaires classiques,
- absence de données bruitées ou fortement incomplètes,
- distribution non exhaustive des tailles et densités possibles.

Ainsi, les conclusions peuvent ne pas s'appliquer directement à des contextes plus atypiques.

Généralisation aux formats

Les expérimentations reposent principalement sur :

- un format CSV standardisé,
- une sortie au format DOT pour Graphviz.

Les résultats pourraient différer pour :

- d'autres formats d'entrée (JSON, bases relationnelles),
- d'autres représentations de sortie (JSON graph, formats propriétaires).

Généralisation aux modèles

Les performances observées sont spécifiques aux modèles testés :

- les résultats peuvent varier avec d’autres architectures,
- les capacités de raisonnement algorithmique diffèrent fortement selon les modèles,
- les effets de prompting influent sur la capacité des LLMs à produire un code correct.

9.3 Biais méthodologiques

Plusieurs choix méthodologiques peuvent influencer les résultats observés.

Sélection des prompts

Les prompts utilisés ont été soigneusement conçus pour maximiser la performance des modèles, ce qui introduit un biais potentiel :

- formulation fortement structurée,
- inclusion de contraintes explicites,
- proximité avec une spécification logique complète.

Cela peut favoriser certains modèles mieux adaptés aux instructions longues et détaillées.

Choix des itérations et versions

Les résultats peuvent dépendre :

- des versions spécifiques des modèles utilisés,
- de la variabilité entre exécutions,
- du nombre limité de répétitions expérimentales.

Choix de la référence (FCA4J)

FCA4J est utilisé comme référence de correction structurelle. Toutefois :

- il constitue une implémentation particulière de FCA,
- certaines différences peuvent relever de choix d’implémentation et non d’erreurs,
- la notion d’équivalence structurelle dépend des critères retenus.

9.4 Synthèse

Les résultats doivent donc être interprétés en tenant compte de plusieurs sources de biais potentiels. Bien que les tendances globales restent robustes, notamment les différences entre familles de modèles et les effets des contraintes mémoire, les valeurs absolues et certains cas limites peuvent être influencés par des facteurs expérimentaux, environnementaux ou méthodologiques.

10 Conclusion et perspectives

Ce travail avait pour objectif d’évaluer la capacité des grands modèles de langage à générer des implémentations correctes et efficaces d’algorithmes d’Analyse Formelle de Concepts, en particulier pour le calcul de treillis de Galois à partir de contextes binaires. Les résultats obtenus montrent qu’il est effectivement possible, à l’aide de prompts suffisamment structurés et contraints, d’obtenir des implémentations fonctionnelles capables de produire des treillis corrects sur des jeux de données réels. Certaines implémentations générées, notamment celles produites par Claude Sonnet 4.6, atteignent un niveau de correction structurelle équivalent à celui de FCA4J sur l’ensemble des jeux de données testés. Ces résultats confirment qu’un LLM peut reproduire correctement des algorithmes complexes de FCA lorsque le problème est précisément spécifié et que les contraintes de sortie sont rigoureusement définies.

Cependant, l’étude met également en évidence plusieurs limites importantes. D’une part, la correction syntaxique d’un programme ne garantit pas sa validité algorithmique. Plusieurs implémentations produisaient des fichiers DOT syntaxiquement valides tout en générant des structures

incompatibles avec un véritable treillis de concepts. D'autre part, les performances observées varient fortement selon les modèles et selon la formulation des prompts. Certains modèles privilégient des solutions rapides mais incorrectes, tandis que d'autres génèrent des implémentations théoriquement plus robustes mais présentant des difficultés pour passer à l'échelle sur des contextes de grande taille.

L'expérimentation menée sur les implémentations avec contrainte mémoire montre également que la simple demande d'une stratégie "memory-aware" ne suffit pas à produire un algorithme réellement scalable. Plusieurs modèles ont proposé des stratégies de partitionnement incomplètes ou incorrectes, ne garantissant ni l'exhaustivité des concepts ni la validité du treillis final. À l'inverse, certaines implémentations ont adopté des stratégies hybrides plus efficaces, exploitant le disque dur pour limiter l'usage de mémoire vive tout en conservant une structure correcte du calcul.

Sur le plan méthodologique, ce travail souligne l'importance du prompt engineering dans l'évaluation des capacités des LLM sur des tâches algorithmiques complexes. Les prompts les plus performants sont ceux qui :

- définissent explicitement le cadre théorique,
- imposent des contraintes fortes sur le format de sortie,
- détaillent les invariants algorithmiques attendus,
- limitent les ambiguïtés d'interprétation,
- imposent une structure de raisonnement progressive.

À l'inverse, les prompts laissant une trop grande liberté de conception conduisent fréquemment à des approximations algorithmiques, à des omissions de concepts intermédiaires ou à des erreurs dans les relations de couverture. Nous émettons tout de même un doute sur la véracité de ces affirmations dans le temps au vu des différences observées entre les modèles les plus performants et les autres modèles au cours de cette étude. Plusieurs perspectives peuvent prolonger ce travail. Tout d'abord, l'évaluation pourrait être étendue à un plus grand nombre de modèles, notamment des modèles open-source récents spécialisés dans le raisonnement algorithmique ou la génération de code scientifique. Une comparaison entre modèles généralistes et modèles spécialisés constituerait également une piste intéressante. Ensuite, le protocole expérimental pourrait être renforcé par l'exécution de multiples générations par prompt afin d'étudier le non-déterminisme des modèles et la stabilité des implémentations produites. Une telle approche permettrait d'évaluer non seulement la qualité moyenne des générations, mais également leur variance.

D'autres pistes concernent l'amélioration des métriques d'évaluation. Les mesures actuelles portent principalement sur :

- la correction structurelle,
- les temps d'exécution,
- l'usage mémoire,
- l'activité disque.

Des métriques complémentaires pourraient être ajoutées, comme :

- la qualité logicielle du code généré,
- la maintenabilité,
- la robustesse face aux entrées invalides,
- la stabilité des performances,
- la consommation énergétique,
- la précision théorique des relations de couverture.

Enfin, une perspective particulièrement intéressante serait l'étude de protocoles hybrides combinant génération par LLM et vérification formelle. Une telle approche pourrait permettre d'utiliser les LLM comme assistants de conception algorithmique tout en garantissant mathématiquement certaines propriétés critiques des structures produites.

Ainsi, ce travail montre que les grands modèles de langage constituent déjà des outils capables de produire des implémentations complexes en Analyse Formelle de Concepts, mais que leur

utilisation fiable nécessite encore un encadrement méthodologique rigoureux, des protocoles d'évaluation stricts et des mécanismes de validation complémentaires.

A Annexe

A.1 Prompt

A.1.1 Prompt sans contrainte

```
=====
R.C.T.F.C - PROMPT LLM
=====
```

--- RÔLE ---

Tu es un expert en algorithmique, en Analyse Formelle de Concepts (AFC / FCA) et en développement C++ moderne.

--- CONTEXTE ---

L'Analyse Formelle de Concepts (Formal Concept Analysis) est une méthode mathématique qui, à partir d'un contexte formel (une relation binaire objets $\times$ attributs), calcule l'ensemble des concepts formels et leur organisation en treillis de Galois. Un concept formel est une paire (extension, intension) où l'extension est un ensemble d'objets et l'intension est l'ensemble de TOUS les attributs partagés par ces objets, et vice-versa. Le treillis représente l'ordre partiel entre ces concepts et est visualisé via le diagramme de Hasse (relation de couverture directe).

--- TÂCHE ---

Écris un programme C++ complet, compilable et fonctionnel, qui :

1. Lit un contexte formel depuis un fichier CSV (chemin passé en argument en ligne de commande).
2. Calcule l'intégralité du treillis de Galois (tous les concepts formels + diagramme de Hasse) en une seule passe algorithmique directe, sans étape intermédiaire manuelle.
3. Produit en sortie un fichier .dot (nommé automatiquement d'après le fichier d'entrée, extension remplacée par ".dot") représentant le treillis au format Graphviz, en respectant exactement les conventions de format décrites ci-dessous.

--- FORMAT ---

Format d'entrée (CSV) :

- Séparateur : point-virgule (;)
- Première ligne : en-tête - première cellule vide, puis les noms des attributs
- Lignes suivantes : nom de l'objet ; puis 1 ou 0 pour chaque attribut

Exemple :

```
;flies;nocturnal;feathered
bat;1;1;0
ostrich;0;0;1
```

Format de sortie (.dot) :

Le fichier doit être un graphe orienté Graphviz avec les règles suivantes :

En-tête :

```
digraph G {  
    rankdir=BT;
```

Nœuds :

Chaque concept formel reçoit un identifiant entier (0, 1, 2, ...) attribué dans l'ordre croissant du nombre d'attributs dans l'intension (puis ordre lexicographique si égalité).

Format de chaque nœud :

```
ID [shape=record,style=filled,COULEUR,label=  
"{ID (I: NB_INTENSION, E: NB_EXTENSION)|ATTRS_NOUVEAUX\n|OBJETS_NOUVEAUX\n}"];
```

Où :

- NB_INTENSION = nombre total d'attributs dans l'intension du concept
- NB_EXTENSION = nombre total d'objets dans l'extension du concept
- ATTRS_NOUVEAUX = liste des attributs "nouveaux" de ce concept (séparés par \n), c'est-à-dire les attributs m dont le concept générateur $\gamma(m) = (\text{closure}(\{m\}), \{m\})$ est exactement ce concept.
Si aucun attribut nouveau : champ vide.
- OBJETS_NOUVEAUX = liste des objets "nouveaux" de ce concept (séparés par \n), c'est-à-dire les objets g dont le concept générateur $\mu(g) = (\{g\}, \text{closure}(\{g\}))$ est exactement ce concept.
Si aucun objet nouveau : champ vide.

Règle de couleur (fillcolor) :

- 'fillcolor=lightblue' si le concept n'introduit AUCUN nouvel objet (OBJETS_NOUVEAUX vide)
- 'fillcolor=orange' si le concept introduit 2 nouveaux objets ou plus ($|\text{OBJETS_NOUVEAUX}| \geq 2$)
- Pas de fillcolor (style=filled suffit) si le concept introduit exactement 1 nouvel objet

Arêtes (diagramme de Hasse - couvertures directes) :

A -> B

signifie que le concept A couvre directement le concept B (A est immédiatement supérieur à B dans l'ordre du treillis).

Les arêtes vont du concept plus spécifique (intension plus grande) vers le concept plus général (intension plus petite), avec rankdir=BT le rendu va de bas en haut.

Pied de fichier :

```
}
```

Exemple de sortie (extrait) :

```
digraph G {  
    rankdir=BT;  
    0 [shape=record,style=filled,fillcolor=lightblue,label="{0 (I: 0, E: 10)|}|}"];  
    1 [shape=record,style=filled,fillcolor=lightblue,label="{1 (I: 1, E: 8)|flies\n|}|}"];
```

```

2 [shape=record,style=filled,label="{2 (I: 2, E: 1)|nocturnal\n|bat\n}"];
6 [shape=record,style=filled,fillcolor=orange,label="{6 (I: 6, E: 3)|
migratory\n|greater-flamingo\n|little-tern\n}"];
    1 -> 0
    2 -> 1
}

```

--- CONTRAINTES ---

1. Un seul fichier source C++ (pas de headers séparés, pas de dépendances externes).
2. Le treillis doit être calculé en une seule passe algorithmique directe et complète (pas d'appel à un outil extérieur, pas d'étapes manuelles intermédiaires). Utilise un algorithme de génération incrémentale ou par fermeture (ex. : Next Closure, Close by One, ou équivalent) implémenté entièrement dans le fichier.
3. Le programme prend exactement un argument : le chemin vers le fichier CSV d'entrée.
4. Le fichier .dot de sortie est créé automatiquement dans le même répertoire que le fichier d'entrée, avec le même nom de base et l'extension ".dot".
5. Le programme doit être robuste : gérer les fichiers introuvables et les formats incorrects avec un message d'erreur clair sur stderr.
6. Code C++17, compilation avec : `g++ -std=c++17 -O2 -o lattice lattice.cpp`

A.1.2 Prompt avec contrainte mémoire

Rôle

Tu es un expert en **Formal Concept Analysis (FCA)**, en **algorithmique Python** et en **conception d'algorithmes économes en mémoire**.

Contexte

Tu dois concevoir un algorithme Python capable de calculer le **treillis des concepts formels** à partir d'un **contexte binaire** stocké dans un fichier CSV.

Le but principal est de **terminer le calcul sans saturer la mémoire vive**, en utilisant une **décomposition du calcul en partitions**, avec écriture intermédiaire sur disque et libération explicite de la mémoire.

Le fichier 'Animals11.csv' n'est **qu'un exemple de format d'entrée**.

Tu ne dois faire **aucune hypothèse spécifique** sur ses objets, ses attributs ou sa taille.

Objectif

Conçois un algorithme Python qui :

1. lit un contexte formel binaire depuis un CSV ;
2. calcule **tous les concepts formels** du contexte ;
3. calcule les **relations de couverture** du treillis ;
4. décompose le calcul en partitions pour réduire l'usage mémoire ;
5. sauvegarde les résultats intermédiaires sur disque ;
6. libère la mémoire entre les partitions ;
7. recompose les résultats ;
8. génère un fichier DOT final nommé :

'Lattice/<nom_du_fichier>_LLM.dot'

Exemple : pour 'Animals11.csv', la sortie attendue est :

```
'Lattice/Animals11_LLM.dot'
```

Contraintes générales

Respecte impérativement les contraintes suivantes :

- L'algorithme doit fonctionner sur des ****CSV de taille variable****, du petit contexte au très grand contexte.
- L'algorithme doit calculer le treillis de ****n'importe quel contexte valide****, pas uniquement celui d' 'Animals11.csv'.
- L'algorithme ne doit ****pas conserver le treillis complet en mémoire**** pendant toute l'exécution.
- L'algorithme doit être ****déterministe****, ****modulaire****, ****lisible**** et ****commenté****.
- L'algorithme doit rester correct même si le nombre d'objets et d'attributs varie fortement.
- Si une hypothèse est nécessaire, énonce-la explicitement avant d'écrire le code.
- Si plusieurs choix techniques sont possibles, choisis celui qui maximise la robustesse et la sobriété mémoire.

Format d'entrée

Le CSV représente un ****contexte formel**** :

- première ligne : attributs ;
- première colonne : objets ;
- cellules : valeurs dans '{0,1}'.

Exemple :

```
'''csv
;flies;nocturnal;feathered
bat;1;1;0
ostrich;0;0;1
'''
```

Interprétation :

- $\$G\$$ = ensemble des objets ;
- $\$M\$$ = ensemble des attributs ;
- $\$I\$$ = relation d'incidence.

Format de sortie

Le programme doit produire un fichier DOT :

```
'Lattice/<nom_du_fichier>_LLM.dot'
```

Structure attendue :

```
'''dot
digraph G {
rankdir=BT;

<définitions des noeuds>

<arêtes>
}
'''
```

Format d'un noeud :

```
'''dot
ID [shape=record,style=filled,label="{ID (I: X, E: Y)|intent|extent}"];
'''
```

avec :

- 'ID' : identifiant du concept ;
- 'I' : cardinalité de l'intent ;
- 'E' : cardinalité de l'extent.

Règles impératives pour les labels

- Affiche dans 'intent' uniquement les **attributs propres au concept**.
- Affiche dans 'extent' uniquement les **objets propres au concept**.
- **Ne recopie pas inutilement** des objets ou attributs dans plusieurs noeuds si leur présence est déjà induite par la structure du treillis.
- Évite absolument une sortie verbeuse où chaque noeud répète toutes les informations héritées.

Règles impératives pour les noeuds intermédiaires

- Génère tous les **concepts nécessaires** au treillis, y compris les concepts intermédiaires servant à relier correctement les autres concepts.
- Ne supprime pas un concept sous prétexte que son label semble "vide" ou partiellement vide.
- Les concepts avec intent vide, extent vide, ou avec peu d'information affichée peuvent être structurellement nécessaires et doivent être conservés s'ils appartiennent au treillis.

Règles impératives pour les couleurs

Attribue les couleurs selon le **nombre d'objets affichés dans le noeud** :

- 'fillcolor=lightblue' si le noeud affiche **0 objet** ;
- **aucune couleur particulière** si le noeud affiche **exactement 1 objet** ;
- 'fillcolor=orange' si le noeud affiche **strictement plus d'un objet**.

Applique ces règles de manière déterministe dans le DOT final.

Stratégie mémoire imposée

Tu dois proposer une stratégie par partitions.

Le principe est le suivant :

1. calculer un sous-ensemble des concepts ;
2. enregistrer ce sous-ensemble sur disque ;
3. libérer la mémoire ;
4. passer à la partition suivante ;
5. recharger et fusionner les résultats à la fin.

Utilise une organisation du type :

```
'''text
partition/
partition/part1/
partition/part2/
...
'''
```

avec des fichiers de travail du type :

'partition/partX/concepts.json'

Exemple de structure JSON :

```
'''json
[
  {
    "intent": ["flies", "feathered"],
    "extent": ["bat"]
  }
]
'''
```

Tâche à accomplir

Rédige d'abord un raisonnement structuré, puis le code Python complet.

Tu dois suivre ****strictement**** les étapes ci-dessous.

Étapes à suivre

Étape 1 - Chargement du contexte formel

Explique précisément :

- comment parser le CSV sans supposer un nombre fixe de lignes ou de colonnes ;
- quelles structures de données utiliser ;
- comment représenter les objets, les attributs et la matrice binaire.

Utilise explicitement :

- une liste d'objets ;
- une liste d'attributs ;
- une matrice binaire ou une structure équivalente clairement justifiée.

Donne la complexité en fonction de $|G|$ et $|M|$.

Étape 2 - Opérateur de fermeture

Explique comment implémenter :

```
$$
closure(X) = X''
$$
```

Décris explicitement :

1. la récupération des objets possédant tous les attributs de XX ;
2. le calcul des attributs communs à ces objets.

Donne la complexité en fonction de $|G|$ et $|M|$.

Étape 3 - Énumération des concepts

Conçois un algorithme de type **NextClosure** ou équivalent, adapté à la FCA.

Explique explicitement :

- l'ordre lexicographique ou lectique utilisé ;
- la génération des intents candidats ;
- l'usage de la fermeture ;
- la condition d'arrêt ;
- pourquoi cette méthode énumère bien tous les concepts requis.

Donne la complexité théorique et les limites pratiques.

Étape 4 - Énumération par partitions

Explique comment découper l'espace de recherche par partitions d'attributs.

Tu dois préciser :

- comment définir une partition ;
- comment choisir la taille d'une partition ;
- comment adapter ce choix à la mémoire disponible ;
- comment éviter les doublons entre partitions ;

- comment garantir que ****tous**** les concepts sont couverts.

Si une simple partition par blocs d'attributs ne suffit pas à garantir l'exhaustivité ou l'absence de doublons, explique le mécanisme complémentaire à ajouter.

Étape 5 - Écriture sur disque

Explique :

- comment créer les dossiers de travail ;
- comment sérialiser les concepts ;
- quel format JSON utiliser ;
- s'il faut écrire en flux ou par lots ;
- comment limiter l'usage mémoire pendant l'écriture.

Étape 6 - Libération mémoire

Explique précisément :

- quelles variables doivent être supprimées après chaque partition ;
- quand utiliser 'del' ;
- quand appeler 'gc.collect()' ;
- quelles structures doivent rester en mémoire et lesquelles doivent être évacuées.

Étape 7 - Boucle principale de partitionnement

Explique le déroulement complet de la boucle :

1. charger ou préparer la partition ;
2. calculer les concepts associés ;
3. enregistrer le résultat ;
4. libérer la mémoire ;
5. passer à la partition suivante.

Montre comment cette boucle permet de traiter des contextes de grande taille.

Étape 8 - Recomposition des partitions

Explique comment :

- relire les partitions ;
- fusionner les résultats ;
- supprimer les doublons ;
- trier les concepts de façon déterministe.

Précise comment conserver une mémoire maîtrisée lors de cette phase finale.

Étape 9 - Calcul des arêtes du treillis

Explique comment calculer la **relation de couverture**.

Un concept A est relié à un concept B si :

```
$$
intent(A) \subset intent(B)
$$
```

et s'il n'existe **aucun concept intermédiaire** entre les deux.

Décris l'algorithme choisi, sa complexité, et la manière de garantir que les concepts intermédiaires nécessaires sont bien présents.

Étape 10 - Génération du DOT

Explique :

- la numérotation des noeuds ;
- l'ordre d'écriture ;
- le format exact des labels ;
- l'application des couleurs ;
- l'écriture des arêtes ;
- les choix assurant une sortie stable et déterministe.

Fonctions minimales à fournir

Le code final doit au minimum contenir les fonctions suivantes :

- 'load_context()'
- 'closure()'
- 'next_closure_partition()'
- 'save_partition()'
- 'load_partitions()'
- 'compute_edges()'
- 'write_dot()'
- 'main()'

Tu peux ajouter des fonctions auxiliaires si cela améliore la modularité.

Exigences de qualité logicielle

Le code final doit être :

- correct ;
- clair ;
- modulaire ;
- commenté ;
- maintenable ;
- robuste face aux cas ambigus ;
- stable sur des tailles d'entrée variables.

Évite les raccourcis fragiles, les hypothèses cachées et les dépendances inutiles.

Format de sortie attendu

Ta réponse doit contenir **exactement** les sections suivantes, dans cet ordre :

1. 'Hypothèses et choix de conception'
2. 'Étape 1 - Chargement du contexte formel'
3. 'Étape 2 - Opérateur de fermeture'
4. 'Étape 3 - Énumération des concepts'
5. 'Étape 4 - Énumération par partitions'
6. 'Étape 5 - Écriture sur disque'
7. 'Étape 6 - Libération mémoire'
8. 'Étape 7 - Boucle principale de partitionnement'
9. 'Étape 8 - Recomposition des partitions'
10. 'Étape 9 - Calcul des arêtes du treillis'
11. 'Étape 10 - Génération du DOT'
12. 'Code Python complet'
13. 'Exemple d'exécution'

Dans la section 'Code Python complet', fournis un script Python exécutable.

Dans la section 'Exemple d'exécution', termine par :

```
'''bash
python lattice.py Animals11.csv
'''
```

et indique la sortie :

```
'''text
Lattice/Animals11_LLM.dot
'''
```

Consigne finale

Réfléchis de manière structurée, puis écris une solution complète.

Ne te limite pas à un simple squelette : fournis un algorithme réellement exploitable, cohérent avec la théorie de la FCA, économe en mémoire, capable de traiter des contextes CSV de taille variable, et apte à produire un treillis complet et correctement relié.

Références

- [1] Anthropic. Prompt engineering overview. URL : <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>.
- [2] Jacob Austin, Augustus Odena, Maxwell Nye, Henryk Michalewski Maarten Bosma, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL : <https://arxiv.org/abs/2108.07732>, arXiv:2108.07732.

- [3] Marc Barbut and Bernard Monjardet. Ordre et classification : algèbre et combinatoire, volume 2 of Méthodes mathématiques des sciences de l'homme. Hachette, Paris, 1970.
- [4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, et al. Evaluating large language models trained on code, 2021. URL : <https://arxiv.org/abs/2107.03374>, arXiv:2107.03374.
- [5] Valentino Cocks, Arame Diop, Oscar Jimenez Flores, Yazmín Mendoza, Marianne Huchard, and Huaxi Yulin Zhang. LLMs Do It All : A Reality Check with Formal Concepts, September 2025. Oral communication at the workshop ConSOFT (Conceptual Knowledge Software : Recent Advancements and Examples), during the 2nd International Joint Conference on Conceptual Knowledge Structures (<https://www.kde.cs.uni-kassel.de/consoft/>). URL : <https://hal-lirmm.ccsd.cnrs.fr/lirmm-05244628>.
- [6] Bernhard Ganter and Rudolf Wille Gerd Stumme. Formal Concept Analysis : Mathematical Foundations, volume 3627 of Lecture Notes in Artificial Intelligence. Springer, 1999. URL : <https://link.springer.com/book/10.1007/978-3-540-31881-1>, doi:10.1007/978-3-540-31881-1.
- [7] Bernhard Ganter and Rudolf Wille. Formal Concept Analysis - Mathematical Foundations, Second Edition. Springer, 2024. doi:10.1007/978-3-031-63422-2.
- [8] Alain Gutierrez, Marianne Huchard, and Pierre Martin. FCA4J : A java library for relational concept analysis and formal concept analysis. In Pablo Cordero and Ondrej Krídlo, editors, Proceedings of the Sixteenth International Conference on Concept Lattices and Their Applications (CLA 2022) Tallinn, Estonia, June 20-22, 2022., Tallinn, Estonia, June 20-22, 2022, CEUR Workshop Proceedings, pages 207–212. CEUR-WS.org, 2022. URL : <https://ceur-ws.org/Vol-3308/Paper17.pdf>.
- [9] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench : Can language models resolve real-world github issues ?, 2023. URL : <https://arxiv.org/abs/2310.06770>, arXiv:2310.06770.
- [10] Sergei O. Kuznetsov and Sergei A. Obiedkov. Comparing performance of algorithms for generating concept lattices. J. Exp. Theor. Artif. Intell., 14(2-3) :189–216, 2002. doi:10.1080/09528130210164170.
- [11] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. arXiv preprint arXiv :2305.01210, 2023. URL : <https://arxiv.org/abs/2305.01210>.
- [12] Mammouth AI. Comment rédiger des prompts efficaces. URL : <https://info.mammouth.ai/fr/docs/how-to-write-an-effective-prompt/>.
- [13] OpenAI. Guide de prompt engineering. URL : <https://developers.openai.com/api/docs/guides/prompt-engineering>.
- [14] Prompting Guide. Prompt engineering guide. URL : <https://www.promptingguide.ai/fr>.
- [15] Shudan Zhang, Hanlin Zhao, Xiao Liu, Qinkai Zheng, Zehan Qi, Xiaotao Gu, Xiaohan Zhang, Yuxiao Dong, and Jie Tang. Naturalcodebench : Examining coding performance mismatch on humaneval and natural user prompts. arXiv preprint arXiv :2405.04520, 2024. URL : <https://arxiv.org/abs/2405.04520>.